

CogACT 动作生成路径蒸馏项目：10 分钟面试讲稿

使用说明：正文按正常语速约 10 到 12 分钟。面试时不需要背公式，重点记住每一段开头的结论、关键数字和因果关系。

一、完整演讲稿

1. 项目背景与动机，约 1 分钟

我这个项目解决的是机器人 VLA 模型的推理效率问题。具体来说，视觉语言模型可以根据图像和指令理解任务，但真正输出连续机器人动作时，CogACT 使用的是 Diffusion Action Model。Diffusion 对连续、多模态动作比较友好，但代价是一次 action chunk 需要反复进行多步去噪，每一步又要完整经过多层 Transformer，所以推理延迟比较高。

我观察到，现有加速方法通常只处理一个维度：要么减少 DDIM 采样步数，要么压缩网络深度；常见蒸馏损失也主要对齐最终动作，或者只对齐几个时间采样点。这样虽然能让 Student 模仿 Teacher 的结果，却没有利用 Teacher 在“去噪时间”和“网络深度”两个维度上逐步形成动作决策的过程。

因此我的核心问题是：能不能把 Teacher 原本稠密的动作生成路径，压缩成 Student 可以学习的少量关键节点，让 Student 不只学最终答案，还学习动作是怎样逐步被修正出来的？

2. 为什么选择 CogACT，约 1 分钟

我选择 CogACT，而不是直接从 OpenVLA 的离散 action token 路线开始，主要有三个原因。

第一，CogACT 把视觉语言理解和连续动作生成拆开了。前面的 VLM 负责从图像和指令中提取 cognition feature，后面的 Action Model 使用 DiT 生成连续 action chunk。这使我可以冻结昂贵的视觉和语言骨干，只蒸馏动作模块，研究边界更清晰，训练成本也更可控。

第二，Diffusion 比单次 MLP 回归更适合连续机器人动作。机器人动作可能存在多种合理轨迹，直接做均方误差回归容易得到平均化动作；Diffusion 从噪声逐步修正动作，更适合表达这种条件分布。DDIM 又提供了确定性的少步采样路径，便于 Teacher 和 Student 使用同一初始噪声进行对齐。

第三，CogACT 的 Action Model 是 DiT。每个 Transformer block 都通过 attention、MLP、残差和归一化逐层更新 action token，因此网络深度本身也包含一条“动作细化路径”。这正好对应我的研究目标：同时压缩时间步和网络深度。

3. 架构与问题定义，约 1 分钟

整个系统可以分为三段。输入是相机图像、自然语言指令和历史动作；先接收视觉数据，由 dinov2 与 siglip 进行生成 visual tokens，然后 llama2 大模型将图像 token 和文本 token 融合，输出一个 cognition feature；Action Model 再以 cognition feature 为条件，从高斯噪声生成长度为 16、每步 7 维的 action chunk，7 维包括位移、旋转和夹爪控制。

Teacher 使用较大的 DiT-B 和更多 DDIM 步数。以 20 个 DDIM step、12 个 Transformer block 为例，每个 action chunk 内部会产生 20 乘 12，也就是 240 个深度动作估计。目标 Student 可以使用 4 个 DDIM step 和 6 层 DiT-S，只保留 24 个节点，理论节点数压缩 10 倍。

Student 也使用 DiT-B，所以是 4 乘 12，也就是 48 个 Student 节点。

4. 核心方法设计, 约 2 分钟

我的方法分为三层。

第一层是暴露 DiT 的中间动作预测。原始 forward 只返回最后一层噪声预测。我增加了可选接口, 在每个 Transformer block 后保存隐藏状态, 并复用同一个 final layer, 将每层隐藏状态投影成噪声预测 `eps_l`。默认 forward 行为保持不变, 所以不会破坏原来的训练和推理调用方。然后我根据当前 `x_t` 和扩散系数, 把每一层的 `eps_l` 转成统一语义的 `x0_l`, 也就是这一层对干净动作的估计。这样不同时间、不同深度的节点都能放在同一个动作空间比较。

第二层是构造并压缩 Teacher 路径。Teacher 的 240 个节点并不是等距的, 有些层动作几乎不变, 有些层修正很大。如果简单每隔 10 个节点采样一次, 会忽略真实的动作变化。因此我根据相邻 `x0` 的动作距离计算累计细化进度, 再归一化到 0 到 1, 最后按进度插值出与 Student 数量相同的锚点。这个方法的含义是: 不是按网络编号对齐, 而是按“完成了多少动作修正”对齐。

第三层是损失设计。`L_task` 保留原始基于 GT action 的 diffusion noise prediction loss; `L_path_teacher` 约束 Student 中间状态; `L_macro_teacher` 对齐相邻节点的动作更新量, 避免 Student 虽然经过相似位置, 但更新方向和节奏不一致; `L_traj_teacher` 对齐 DDIM 时间轨迹。`L_final_gt` 监督全部 DDIM 去噪后的最终 action chunk, 形状都是 [B, T, C], 例如 [B, 16, 7]。

后来我进一步发现, 只让 Student 对齐 Teacher 的最终动作会继承 Teacher 偏差, 所以做了 Teacher 和 GT 的职责拆分: Teacher 负责监督过程, GT 负责监督 Final Action。具体实现中, `L_final_gt` 直接对齐数据集真实动作, 而 Teacher 的最终节点在轨迹损失和路径压缩之前就被排除。这里“压缩之前排除”很重要; 如果先用完整 Teacher 路径做进度归一化, 再删除最后一个锚点, Teacher 终点仍然会改变前面所有锚点的位置, 职责并没有真正分开。

5. 方案落地与训练 Pipeline, 约 1.5 分钟

我复用了 CogACT 的 Open-X 数据接口, 当前使用 BridgeData。底层通过 RLDS 和 `tf.data` 读取 episode, 再经过图像变换、tokenizer、prompt builder 和 action window 切分, 最后由 collator 组装 batch。一个 batch 里主要包含 `pixel_values`、`input_ids`、`attention_mask` 和 action label。训练时取最后 16 个未来动作作为监督目标。分布式场景下, 因为这里的数据集是 IterableDataset, 按 sample index 对 world size 取模的 rank 分片, 防止不同 GPU 重复读取同一批样本。

Teacher 加载后设置为 eval, 并关闭所有参数梯度。每个 batch 先经过冻结的 VLM 得到 cognition feature。Teacher 使用 conditional 和 unconditional 两个分支做 classifier-free guidance, 并从与 Student 相同的初始噪声开始运行 DDIM。采样过程中我同时保存三类信息: 最终动作、与 Student 时间步对齐的 trajectory, 以及每个 DDIM step 内所有 Transformer block 的 `x0` 深度路径。Teacher 全程放在 `torch.no_grad()` 中, 避免存储反向图。

Student 使用相同的 cognition feature 和初始噪声, 只运行更少的 DDIM step, 并记录自己的时间轨迹和深度路径。随后计算 GT final、Teacher trajectory、Teacher path、macro update 和基础 diffusion loss。反向传播只更新 Student Action Model, Teacher 和 VLM 不参与梯度更新。

训练循环包括 forward、loss、backward、gradient clipping、AdamW update、checkpoint 和周期性 validation。我使用 warmup 加学习率调度, 当前 base learning rate 是 $1.4e-6$, 梯度裁剪阈值是 10。训练支持从 checkpoint 恢复 Student、optimizer、epoch 和 global step; 日志同时写本地 JSONL 和 W&B, 记录各项 loss、梯度范数、节点数量以及 Student 对 GT、Student 对 Teacher、Teacher 对 GT 三种误差, 便于判断到底是 Student 没学会, 还是 Teacher 本身和 GT 存在偏差。

代码组织上，我把职责拆得比较清楚：`loaders.py` 管 Teacher、Student 和数据加载；`runners.py` 管 cognition feature、Teacher/Student DDIM 以及轨迹记录；`path.py` 管细化进度、插值和路径损失；`loss.py` 只组合训练目标；`train.py` 管训练、checkpoint、验证和日志；`eval_distillation.py` 做可复现的离线评估。这样扩散数学、路径算法和训练基础设施是解耦的，便于分别做单元测试和消融。

6. 分布式训练与显存工程，约 1.5 分钟

实际训练使用两张 GPU 和 DDP，每卡 batch size 为 4，全局 batch size 为 8。每个 rank 持有一份 Student 参数并处理不同数据，反向时 DDP 把梯度按 bucket 交给 NCCL 做 AllReduce，因此各卡更新后仍保持同一份模型。这里使用 DDP 的目标是提高吞吐并扩大 global batch，不是解决单卡放不下模型。（回答为什么 batch 不能设置太小，甚至有时候比单卡还慢？小 batch 时，如果单步计算太短，AllReduce 的通信占比会变高，DDP 甚至可能得不偿失，所以不能只看 GPU 数量，需要看 samples/s 和通信占比。）

（回答FSDP是什么，为什么最终没有做？FSDP 可以分片参数、梯度和 optimizer state，确实适合模型本身放不下的场景；但当前只训练 Action Model，VLM 和 Teacher 都冻结，FSDP 的显存收益不足以抵消参数 all-gather、分片 checkpoint 和设备管理复杂度。我前期还遇到过 CPU offload 与手写 `.to(cuda)` 的设备一致性风险，因此本轮最终选择 DDP，这是基于当前训练对象规模做的取舍。）

显存分析上，训练显存主要来自模型参数、Student 梯度、AdamW 的一阶二阶状态和激活；这个项目还有额外的 Teacher/VLM 参数，以及 240 个中间 x0 节点。Teacher 虽然没有梯度和 optimizer state，但仍占参数显存；Student 的中间节点需要保留反向图，是路径蒸馏新增显存的主要来源。我的处理包括冻结 Teacher、使用 no-grad、bf16、只训练 Action Model、控制 per-device batch，以及增加分阶段显存日志来定位 OOM。相比语言模型自回归推理，这里没有不断增长的 KV cache，主要压力是激活和路径张量。

当前代码还没有实现 gradient accumulation。梯度累计是计算实现大batch的实现方式，在我们这里没有什么实现价值。

7. 实验结果、测试与失败分析，约 1 分钟

我完成了多轮两卡训练和 SimplifierEnv rollout。结果显示：普通 DiT-S 和 DiT-B 减少计算后性能明显下降；加入中间路径监督后，Depth-DiT-B 在三个任务上都优于普通 DiT-B，Teacher/GT 分工后进一步提高。

普通 Student 在复杂交互任务上性能明显下降，例如 Put in Drawer 成功率从 Teacher 的 53.4% 降至 DiT-B 的 7.9%。加入深度路径监督后，Depth-DiT-B 在 Pick Coke 上达到 96.3%，在 Put in Drawer 上从 7.9% 提升至 33.3%，说明过程监督能明显改善动作生成。进一步采用 Teacher 监督过程、GT 监督终点后，Put in Drawer 提升至 46.0%，表明主要改进发生在接触和持续交互阶段。

离线训练曲线也支持这个判断：Student 对 Teacher 的 final MSE 在 step 8,000 达到最好值 0.040676，但之后继续训练反而回升，因此我使用 early stopping 保存 best checkpoint。这个现象说明离线 MSE 和多个路径 loss 不能替代 rollout；最终仍要按任务阶段分析模型究竟失败在感知、接触还是动作执行。

8. 下一步工作，约 1 分钟

下一步我会按三个优先级推进。

第一，补齐严格消融。在相同数据、seed 和训练预算下分别比较 final、trajectory、path、macro，以及 Teacher Final、软分工和严格 GT Final，量化每一项对 rollout 成功率的独立贡献。

第二，完成真正的深度压缩，把 Student 从当前 DiT-B 的 48 节点切换到 DiT-S 的 24 节点，并测量吞吐、单 chunk 延迟和成功率之间的 Pareto trade-off。我还会检查共享 final layer 投影中间特征是否产生分布偏差；如

果偏差明显，可以尝试轻量的 layer-specific adapter，但要控制额外参数和推理开销。

第三，增强指令敏感性。未来会加入视觉语言控制贡献蒸馏：分别保留和屏蔽视觉语言条件，计算其宏动作更新的变化量，以此进一步加强语言对动作的指导。

二、每部分核心观点

1. Motivation

一句话：Diffusion Action Model 的成本来自“多时间步乘多网络层”，现有蒸馏没有充分利用这条二维生成路径。

必须记住：不是为了单纯减参数，而是为了降低 action chunk 的采样延迟，同时尽量保持闭环控制能力。

2. 为什么是 CogACT

一句话：CogACT 将 VLM 理解与连续动作生成解耦，便于冻结大骨干、专注蒸馏 DiT Action Model。

必须记住：Diffusion 适合连续、多模态动作；DDIM 提供确定、可对齐的少步路径。

3. 架构

一句话：Teacher 的时间和深度共同形成稠密动作细化路径。

必须记住：设计目标 240 到 24；当前受控实验实际是 240 到 48。

4. 方法

一句话：先把每层噪声预测转换到统一 x_0 动作空间，再按动作变化进度压缩 Teacher 路径。

必须记住： L_{path} 对齐状态， L_{macro} 对齐更新；Teacher 管过程，GT 管终点；终点必须在压缩前排除。

5. 方案落地

一句话：项目已经形成数据、Teacher 采样、Student loss、checkpoint、固定验证缓存和 W&B 的完整闭环。

必须记住：数据、runner、路径算法、loss、train、eval 各自解耦，便于测试和消融。

6. 分布式与显存

一句话：两卡 DDP 是吞吐选择，FSDP 是显存选择；当前只训练 Action Model，所以 DDP 更合适。

必须记住：每卡 4、全局 8；Teacher no-grad；bf16；Student 路径激活是新增显存重点；gradient accumulation 尚未实现。

7. 结果

一句话：Depth-DiT-B 在三个 rollout 任务上都优于普通 DiT-B，其中 Pick Coke 达到 96.3%，Put in Drawer 从 7.9% 提升到 46.0%。

必须记住：诚实区分工程验证、离线指标和闭环成功率。

8. 下一步

一句话：先补齐严格消融，再完成 DiT-S 24 节点，最后研究 Negative Instruction。

三、面试官可能追问的问题

Motivation 与模型选择

1. 为什么不直接量化或者剪枝？
2. 为什么不只减少 DDIM step，网络深度蒸馏有什么额外价值？
3. 为什么 OpenVLA 的离散动作不适合你的方案？
4. Diffusion 相比行为克隆 MLP 的优势如何验证？
5. DDIM 为什么适合 Teacher-Student 对齐，和 DDPM 有什么区别？
6. CogACT 的 VLM 是否也应该一起蒸馏？
7. 冻结 VLM 会不会限制 Student 的上限？

回答抓手：项目先隔离 Action Module 变量；量化、剪枝和本方法正交；DDIM 是确定性少步采样，更易共享 noise 和对齐轨迹。

路径蒸馏方法

1. 中间 block 的隐藏状态为什么可以直接复用最终 `final_layer`？
2. `eps_l` 转换成 `x0_l` 的公式是什么？
3. 同一个 DDIM step 内的深度节点是不是实际采样状态？
4. 为什么不用均匀层索引采样？
5. 动作距离如何处理位移、旋转和夹爪的量纲差异？
6. 为什么同时需要 `L_path` 和 `L_macro`？
7. 插值锚点不是真实 Teacher 节点，会不会引入伪监督？
8. 为什么 Teacher 终点必须在路径压缩前删除？
9. GT Final 会不会损害动作多模态性？
10. 多个 loss 之间如何确定权重？

回答抓手：深度 `x0_l` 是每层对同一 `x_t` 的动作估计，不是 DDIM 真正走过的状态；进度压缩按动作变化而非层编号对齐；量纲问题已预留 action dimension weights；最终判断依赖消融和 rollout。

数据与 Pipeline

1. Open-X 和 BridgeData 的数据格式是什么？
2. action chunk 如何从 episode 中切分？
3. 图像、文本和动作如何对齐？
4. IterableDataset 在 DDP 下如何避免重复样本？
5. 为什么使用 shuffle buffer，大小如何选？
6. 固定 eval cache 解决了什么问题？
7. Teacher cache 为什么没有全部离线预计算？
8. 数据加载是否成为瓶颈，如何证明？

回答抓手：当前在线 Teacher 保证随机增强和条件一致，但计算昂贵；固定 eval cache 固定 batch 与 noise，使 checkpoint 比较可复现；吞吐优化前应先 profile。

分布式与显存

1. DDP 如何进行梯度同步？
2. 为什么每卡 batch 4、全局 batch 8？
3. 小 batch 下 DDP 为什么可能加速不明显？
4. NCCL AllReduce 的通信量与什么相关？
5. FSDP 和 DDP 的显存、通信差别是什么？
6. 为什么项目最后使用 DDP 而不是 FSDP？
7. bf16 为什么比 fp16 更稳定？
8. OOM 时你如何定位是参数、激活还是数据造成的？
9. Gradient accumulation 如何与 DDP `no_sync()` 配合？
10. PCIe、NVLink 和禁用 P2P 会怎样影响性能？

回答抓手：DDP 每卡完整参数，反向按 bucket AllReduce；FSDP 分片参数、梯度、optimizer state，但增加 gather 和 checkpoint 复杂度；当前只训 Action Model，所以选择稳定性更好的 DDP。当前运行环境禁用了 P2P 和 IB，这是稳定性配置，但可能牺牲通信性能，需要单独 benchmark。

训练稳定性与评估

1. 为什么学习率这么小？
2. warmup 解决什么问题？
3. 梯度裁剪阈值 10 如何确定？
4. 为什么 best checkpoint 在 8,000 step，之后反而变差？
5. early stopping 为什么监控 Student-Teacher MSE，而不是 GT MSE？
6. total loss 下降是否代表机器人成功率上升？
7. 如何检测末端动作跳变和动作抖动？
8. 如何保证 resume 后实验一致？
9. 哪些随机状态目前还需要进一步保存？
10. 如何设计 A/B/C 消融才公平？

回答抓手：后期回升可能来自过拟合或 loss 冲突；现有 early-stop metric 仍偏 Teacher，是下一轮应调整的；应同时保存 Python、NumPy、Torch、CUDA 和 sampler 状态以增强严格复现。

机器人闭环与未来工作

1. 为什么离线 MSE 不能代表 success rate？
2. 如何在 SimplerEnv 中做公平评估？
3. action latency 降低多少才有实际价值？
4. DiT-S 容量不足时怎么办？
5. Negative Instruction 的正负样本如何构造？
6. 如何避免 Student 只学 Teacher 的错误？
7. 真实机器人部署还缺哪些工作？

回答抓手：闭环会累积动作误差；需要固定场景、初始条件和 episode 数，报告均值和置信区间；真实部署还要考虑相机延迟、控制频率、安全约束和 distribution shift。

四、容易讲得太论文化的地方与修改建议

1. 不要从公式开始。先说“Teacher 有 240 个决策节点，Student 只能保留 24 个，我需要决定保留哪些过程信息”，再解释进度压缩。
 2. 不要逐层讲 Transformer 结构。只需说明 block 会逐层修正 action token，因此深度维度可视为动作细化过程。
 3. 不要说“提出了一种新型路径参数化方法”。改成“均匀抽样会漏掉修正大的区间，所以我按动作变化量重新参数化路径”。
 4. 不要只罗列五个 loss。把它们归为三类：GT 基础训练、Teacher 过程监督、GT 最终监督。
 5. 不要笼统说“实验显著证明有效”。应直接给出三个 rollout 结果，同时承认 Move Near 只提升 2.91 个百分点，严格消融仍待补齐。
 6. 不要用大量 Related Work 证明选题。面试官更关心你观察到了什么系统瓶颈，以及为什么当前设计能验证这个判断。
 7. 讲工程时不要背 DDP、FSDP 定义。要结合本项目说：为什么只训练 Action Model、为什么 Teacher no-grad、为什么两卡 DDP、为什么 FSDP 暂时收益不够。
 8. 不要掩盖 24 与 48 节点的差异。主动说明当前实验先固定网络深度，下一阶段再切 DiT-S，这反而体现实验变量控制意识。
-

五、体现 AI Infra 能力的内容

- 将 IterableDataset 按 rank 分片，避免 DDP 数据重复。
 - 理解 DDP AllReduce、global batch 与通信开销的关系。
 - 根据训练对象规模在 DDP 和 FSDP 之间做取舍，而不是盲目使用更复杂方案。
 - 使用 bf16、Teacher no-grad、冻结骨干和 batch 控制降低显存。
 - 区分参数、梯度、optimizer state、activation 和路径张量的显存来源。
 - 建立 checkpoint resume、best checkpoint、early stopping、JSONL 和 W&B 监控。
 - 使用固定 batch 与固定 noise 的 eval cache 提高 checkpoint 比较的可复现性。
 - 认识到 `num_workers=0`、shuffle buffer 初始化、Teacher 在线推理和 NCCL 都可能成为吞吐瓶颈，并提出 profiler 驱动优化方法。
 - 能准确说明 gradient accumulation 当前未实现，以及正确接入 DDP 时需要 `no_sync()`。
-

六、体现研究能力的内容

- 从“采样步数乘网络深度”的二维计算结构中提出研究问题。
 - 识别只对齐最终动作会丢失 Teacher 形成动作的过程信息。
 - 将不同层的噪声输出统一转换到 `x0` 动作空间，使比较具有明确语义。
 - 发现均匀节点采样不合理，并用累计动作变化构造细化进度。
 - 区分状态对齐和更新量对齐，设计 `L_path` 与 `L_macro`。
 - 发现 Teacher Final 与 GT Final 的监督职责冲突，并严格在压缩前排除 Teacher 终点。
 - 对 GT 噪声、多模态性、末端跳变、量纲差异和插值伪监督保持审慎。
 - 不把离线 MSE 等同于 rollout success，能够提出公平消融和失败验证方案。
-

七、体现工程能力的内容

- 在保持默认 forward 行为不变的前提下扩展 DiT 中间输出接口。
- 打通 `eps_l` -> `x0_l` -> 时间深度路径 -> 压缩锚点 -> `loss` -> `backward` 全链路。
- 将模型加载、采样 runner、路径算法、loss、训练和评估拆成独立模块。

- 针对 shape、时间步映射、CFG batch 拆分、NaN 和梯度流增加运行时检查。
- 为默认行为、shape、CFG、扩散公式、路径插值、单节点退化和 Teacher/GT 职责边界补充单元测试。
- 完成两卡训练、周期验证、early stopping、best checkpoint 和最终 checkpoint 输出。
- 保留兼容开关，使旧 baseline 可以继续运行，降低实验代码迭代风险。

八、演讲评分与不足

当前评分：86 / 100

优势：研究问题清楚，方法设计有因果链条；工程实现完整，能够自然体现模型、分布式、显存、数据和评估能力；对未完成结果保持诚实，可信度较高。

主要扣分点：

1. A/B/C 以及软分工、严格分工消融尚未完整，暂时不能量化 `L_path`、`L_macro` 和 GT Final 各自贡献。
2. 目标 Student 是 DiT-S 24 节点，但当前完整路径实验仍以 DiT-B 48 节点为主，联合深度压缩收益尚未充分验证。
3. Move Near 只从 70.42% 提升到 73.33%，路径监督对不同任务的收益不均衡。
4. 缺少精确的 latency、samples/s、GPU utilization 和 peak memory 对比，AI Infra 效果还主要停留在工程机制层面。
5. 当前 early stopping 使用 Student-Teacher final MSE，与“GT 监督最终动作”的严格分工目标不完全一致，下一轮应改为 GT 指标或 rollout proxy，并同时监控多项指标。
6. 当前 `num_workers=0`，也没有 gradient accumulation；数据吞吐和 effective batch 仍有明确优化空间。

如果补齐严格消融、DiT-S 路径训练以及延迟和显存数据，这段项目介绍可以提升到 92 分以上。

九、面试前三天的专项准备建议

总体判断

不要只看概念文章，也不要为了“用过”而仓促改动主训练代码。三天内最有效的方法是：显存分析回到本项目做真实测量，Gradient Accumulation 用最小实验验证机制，再把两者组织成可以回答追问的工程决策。

建议时间分配：70% 围绕自己的项目，20% 写和跑最小实验，10% 阅读 PyTorch 官方资料。面试官通常会继续追问“你在项目中看到了什么数据、为什么这样选”，只会复述定义很容易被识别。

第一天：把显存讲清楚并测出来

目标不是背“参数、梯度、优化器、激活”五个名词，而是能解释本项目每个训练阶段的显存变化。

建议在单卡、`batch_size=1` 下记录以下位置的 `memory_allocated`、`memory_reserved` 和 `max_memory_allocated`：

1. Teacher 和 Student 加载完成后。
2. batch 搬到 GPU 后。
3. Teacher no-grad DDIM 和 240 节点记录后。
4. Student forward 和 48 个路径节点后。
5. backward 后。

6. optimizer step 和 `zero_grad(set_to_none=True)` 后。

然后只改变一个变量重复测量：bf16 开关、是否记录深度路径、batch size 1/2/4。最终整理一张小表，能够回答“新增路径蒸馏到底多占多少显存”“显存是否随 batch 近似线性增长”“峰值发生在 forward 还是 backward”。这比阅读多篇泛化文章更有面试价值。

还要能解释三个常见现象：`reserved` 大于 `allocated` 不等于泄漏，它可能来自 CUDA caching allocator；OOM 可能来自真正容量不足，也可能来自碎片；`torch.cuda.empty_cache()` 不能释放仍被 Tensor 引用的显存，也不会降低当前活跃 Tensor 的占用。

第二天：用最小实验掌握 Gradient Accumulation

先不要修改本项目主训练链路。写一个几十行的两卡 DDP 小实验，比较下面两组参数更新是否接近：

- 直接使用 batch size 8 做一次更新。
- micro-batch 2、累计 4 次，每次 loss 除以 4，最后再 optimizer step。

需要亲自验证四件事：只在累积周期开始时 `zero_grad`；loss 要除以 accumulation steps；只在周期结束时 `optimizer.step()`；DDP 的前三个 micro-step 使用 `model.no_sync()`，最后一步才同步梯度。再观察去掉 `no_sync()` 后结果仍可能正确，但通信次数从一次变成四次。

面试时可以诚实表达：当前项目每卡 batch 4 可以稳定训练，所以我没有为了堆技术而接入 accumulation；如果未来 DiT-S/路径激活仍导致 micro-batch 过小，我会用 accumulation 提高 effective batch，并同时重新评估学习率、BatchNorm 类状态和尾部不足一个累积周期的处理。CogACT 的 DiT 主要使用 LayerNorm，不依赖跨 batch 统计，这一点比 BatchNorm 模型更适合 accumulation。

第三天：形成口述闭环，不再扩展功能

用下面四个问题做模拟追问，每题控制在 60 到 90 秒：

1. 你的项目为什么 OOM，峰值在哪里，证据是什么？
2. DDP、FSDP 和 Gradient Accumulation 分别解决什么问题？
3. accumulation 为什么不等于增大 GPU 吞吐，它的主要价值是什么？
4. 如果给你 8 张卡，你如何选择 per-device batch、accumulation steps 和 global batch？

最后准备一句清晰结论：DDP 主要扩展数据并行吞吐，FSDP 主要降低模型状态显存，Gradient Accumulation 主要在显存受限时扩大 effective batch；三者可以组合，但通信、学习率和训练稳定性需要一起评估。

是否要在本项目实现

显存分析：建议立刻在本项目实测。这是低风险、证据密度最高的准备。

Gradient Accumulation：当前不建议为了面试直接合入主训练代码。先完成最小 DDP 实验并读懂实现；只有在前两天还有半天以上余量、且可以补单元测试和 20 到 100 step 短跑时，才增加为默认关闭的可选配置。三天后面试，稳定理解和可解释数据比一个未经充分验证的新开关更重要。